

SOURCE CODE

SPRING BOOT:

```
spring.datasource.url=jdbc:mysql://localhost:3306/campus_event_db
```

```
spring.datasource.username=root
```

```
spring.datasource.password=
```

```
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.show-sql=true
```

```
spring.thymeleaf.cache=false
```

```
server.port=8080
```

JAVA:

```
package com.campus.eventmanagement.entity;
```

```
import jakarta.persistence.*;
```

```
import jakarta.validation.constraints.NotNull;
```

```
import jakarta.validation.constraints.Size;
```

```
@Entity
```

```
@Table(name = "events")
```

```
public class Event {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    @NotNull
```

```
@Size(min = 3, max = 100)
private String eventName;

private String department;

private String eventType;

private String eventDate;

private String venue;

private int seats;

// Getters and Setters
}

package com.campus.eventmanagement.repository;

import com.campus.eventmanagement.entity.Event;
import org.springframework.data.jpa.repository.JpaRepository;

public interface EventRepository extends JpaRepository<Event, Long> {
}

package com.campus.eventmanagement.service;

import com.campus.eventmanagement.entity.Event;
import com.campus.eventmanagement.repository.EventRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
```

```
import java.util.List;
```

```
@Service
```

```
public class EventService {
```

```
    @Autowired
```

```
    private EventRepository repository;
```

```
    public List<Event> getAllEvents() {
```

```
        return repository.findAll();
```

```
    }
```

```
    public Event saveEvent(Event event) {
```

```
        return repository.save(event);
```

```
    }
```

```
    public void deleteEvent(Long id) {
```

```
        repository.deleteById(id);
```

```
    }
```

```
}
```

```
@PostMapping("/save")
```

```
public String saveEvent(
```

```
    @Valid @ModelAttribute Event event,
```

```
    BindingResult result) {
```

```
    if(result.hasErrors()) {
```

```
        return "add-event";
```

```
    }
```

```

        service.saveEvent(event);

        return "redirect:/";
    }

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/add").authenticated()
                .anyRequest().permitAll()
            )
            .formLogin();

        return http.build();
    }
}

```

HTML:

```

<!DOCTYPE html>

<html xmlns:th="http://www.thymeleaf.org">

<head>

    <title>Campus Events</title>

```

</head>

<body>

<h1>Upcoming Events</h1>

Add Event

<table border="1">

<tr>

<th>Name</th>

<th>Department</th>

<th>Date</th>

<th>Action</th>

</tr>

<tr th:each="event : \${events}">

<td th:text="\${event.eventName}"></td>

<td th:text="\${event.department}"></td>

<td th:text="\${event.eventDate}"></td>

<td>

<a th:href="@{/delete/{id}(id=\${event.id})}">

Delete

</td>

</tr>

</table>

</body>

</html>

